

Sensor-Integrated Head Design for Robust Perception in Humanoid Robots

Ana Jaramillo, Summer Undergraduate Research Fellow (SURF) proposal in the laboratory of Prof. Aaron Ames

Introduction

Perception is a key aspect of robotic operation, particularly in humanoid systems. The Unitree G1 humanoid robot currently utilizes a single Intel RealSense D435 RGB-D camera (angled downwards) and one LiDAR sensor. While this configuration provides some special mapping, it could be improved by expanding the visual sensing design to a new robotic head that accommodates additional perception hardware. The key sensor qualities were identified as:

- Internal Measurement Unit (IMU) Integration for real-time pose estimation
- Moderate depth range (3-6 meters)
- Wide field of view (FoV) to reduce blind spots
- ROS 2 compatibility to integrate easily with Unitree G1 software

Our approach begins with validating different multi-camera configuration such as triangular or rectangular on a test platform—without the full design of the head. The prototype will allow us to optimize for visual coverage and physical integration. We will then see if solely a camera-based designed is enough and reevaluate the use of LiDAR. The final stage will involve refining the robot's head shape to more closely resemble a human-like appearance.

The goal of this project is to design a new head for the robot that incorporates all the new sensors allowing for improved perception. To approach this, we will start by trying out 3-4 cameras configurations and seeing how the computers hold up with the data. We will then design a head that correctly accommodates for all the cameras. After seeing this performance, we will see if there are any modifications that need to be made. And finally, after all hardware is determined, we will focus on designing a more human-like head.

Camera Integration

After selecting the Intel RealSense D455 cameras, the hardware environment was set up on the NVIDIA Jetson Orin NX Super. This included installing ROS2 Humble, the Intel RealSense SDK 2.0, and the ROS2 wrapper. After following the installation process, all components were successfully configured, and the cameras were confirmed to be working, with point clouds visualized in RViz2 (Figure 1).

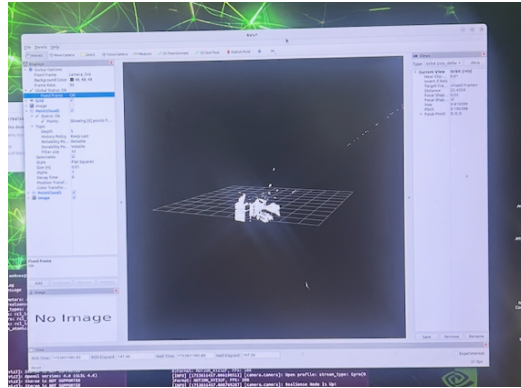


Figure 1. Point cloud visualization of image by Intel RealSense D455 camera through RViz2.

Since three cameras were used, it was necessary to visualize all point clouds simultaneously. To streamline this process, I developed a custom multicamera launch file (`mymultiple_camera_launch.py`), extending the existing multi-camera launch provided by the RealSense ROS package [1]. This launch file was edited to support up to four cameras and allowed all three to be started and visualized at once.

Mechanical Design of Camera Holder

In parallel, I designed a custom camera holder to physically mount the sensors in a configuration that optimized field of view. The first prototype was a four-way camera holder, designed in SolidWorks and 3D-printed (Figure 2) in PLA. This was a crucial step because a rigid, well-defined mount was required to accurately determine the geometric transformations for each camera, ensuring that the point clouds aligned correctly in the robot's coordinate frames.

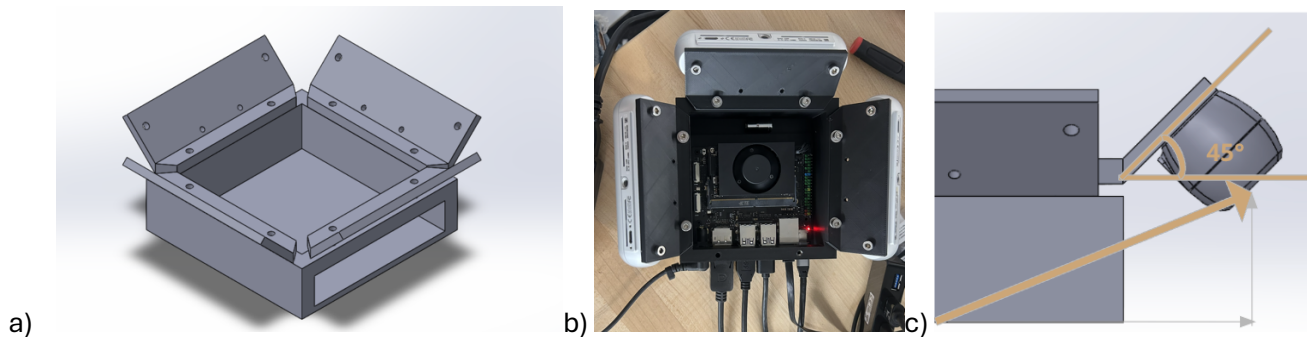


Figure 2. a) SolidWorks CAD design for 4-camera holder. b) 3D printed mount fully assembled and equipped with all three cameras and computer. c) Schematic for transformations needed for successful data orientation.

Once the holder was fabricated and the cameras were mounted, I measured the position and orientation of each camera relative to an arbitrary origin located at the center of the holder's base. Distances in the x, y, and z directions, as well as roll, pitch, and yaw, were recorded. These values were then incorporated into the launch file as static transformations, allowing the point clouds to be displayed in RViz2 with realistic alignment to the cameras' true orientations.

Static Heightfield Visualization in MuJoCo

After achieving multi-camera point cloud visualization in RViz2, the next objective was to represent the data as a MuJoCo heightfield. MuJoCo was installed on my personal computer, where I created a static heightfield from point cloud data published on a ROS2 topic. The script subscribed to the topic, cropped the data to fit a chosen grid size, and mapped each grid cell to a single height value by averaging the z-coordinates of all points within the cell. The resulting values were normalized to produce the grayscale PNG image required by MuJoCo. Using this PNG as input, a heightfield was successfully visualized through the *geom hfield* feature (Figure 4).

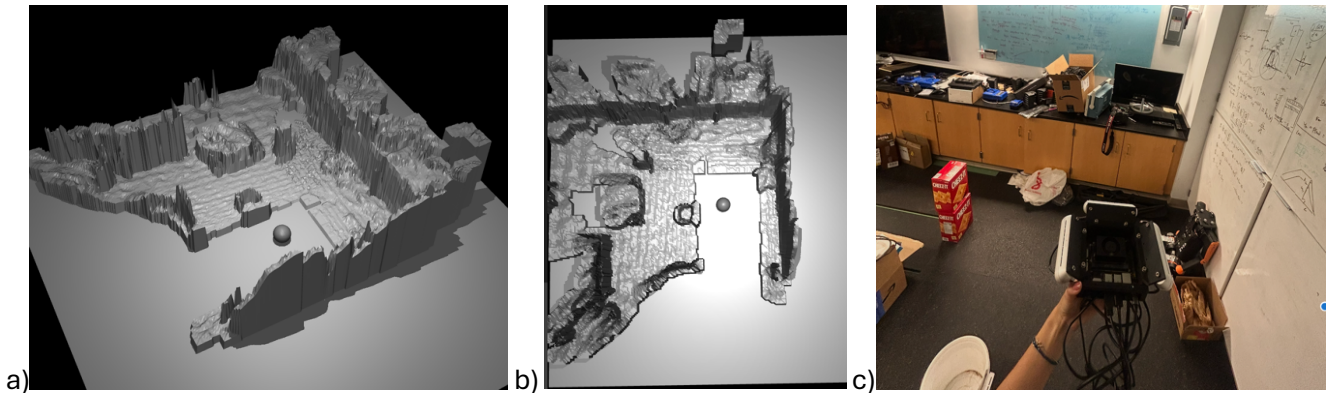


Figure 4. a,b) Same MuJoCo heightfield visualization, different viewer angles. c) Real life point of view for the heightfield produced

Toward Real-Time Heightfields

Although this static PNG-based method demonstrated the feasibility of point cloud to heightfield visualization, it is not practical for mobile robots, as it does not process live data or update in real time. The next step was therefore to generate dynamic heightfields from streaming point clouds. As a first test, I used simulated point cloud data to produce heightfield visualizations.

I then moved on to using actual point cloud data from the cameras (Figure 5), and I am currently working on improving the visualization to correctly simulate the robot's environment. This process has revealed important considerations. Because point cloud data is computationally heavy, every step in the pipeline can introduce latency. For the system to be useful, visual data must be updated at realistic speeds—at least 30 Hz—to ensure the robot can act on it in real time. I have also observed that coordinate transformations have a major impact, particularly in visual research, as they determine how the world is oriented and how the robot interprets its sensor data.

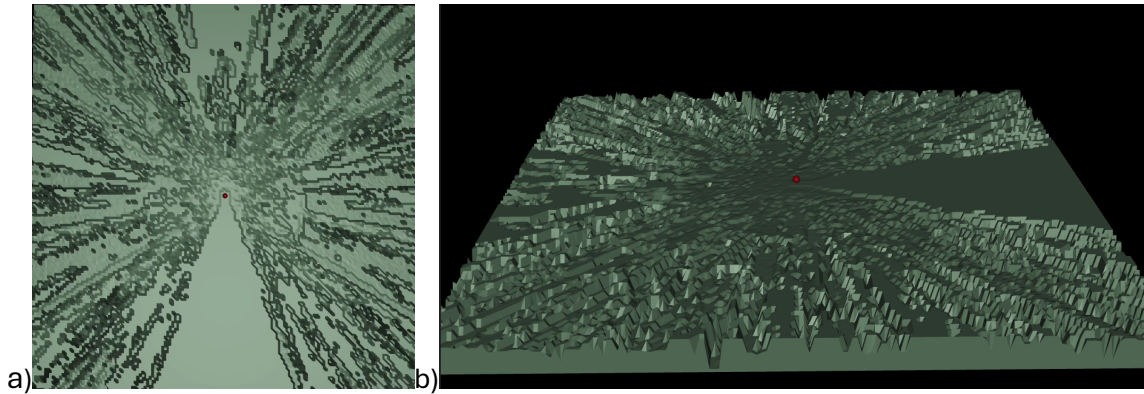


Figure 5. a) Top view of simulated heightfield. b) Side view of simulated heightfield where red ball indicates the mount's location.

Current Challenges

Current challenges involve live point cloud updating. Currently the heightfield does not correctly update or accurately represent the environment around the cameras. Furthermore, current high data transfer between devices as well as the microcomputer may serve as potential bottlenecks in processing. These aspects many prevent real time operation.

Remaining Research Goals

Looking forward, the heightfield visualizations must be improved to update dynamically and accurately. The head design must also be improved by including movable camera angles or mechanisms allowing for head rotation. Finally, transforms and visualizations must be optimized to allow higher speeds and achieve real-time operation at a minimum of 30 Hz.

Notably, the goals of the project have shifted from the original plan. While the initial focus was solely on building a new head design, the work has evolved toward developing a simulation and visualization pipeline. This shift has opened new opportunities for evaluating sensor configurations and perception strategies before deploying them on the physical robot.

References

- [1] "IntelRealSense/realsense-ros: ROS Wrapper for Intel(R) RealSense(TM) Cameras". Consultado: el 1 de septiembre de 2025. [En línea]. Disponible en: <https://github.com/IntelRealSense/realsense-ros/tree/ros2-master>